JAVA Project Report

(CIC-258)

Submitted in partial fulfillment of the
requirement for the award of the degree of

**Bachelor of
Technology in**

Information Technology

Submitted by:

NITIN (51)

SHARAD (39)

OM (40)

MAAN(58)

IT-2

Under the Supervision of

Mr surrender singh & Ms. Nidhi Goel
Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY
MAHARAJA SURAJMAL INSTITUTE OF TECHNOLOGY

C4, JANAKPURI, NEW DELHI – 110058

(GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY, DWARKA,
NEW DELHI)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Ms. Nidhi Goel for giving me the opportunity to work on this project and for her continuous guidance throughout its development. Her valuable support and suggestions helped me understand the concepts more clearly and improve my project successfully. This project gave me a great opportunity to apply my knowledge of Java programming in a practical way.

While working on the project "Password Strength Checker", I was able to understand how basic concepts like object-oriented programming, GUI development, and logical implementation can be used to solve real-life problems. This project also enhanced my problem-solving skills and technical understanding.

I would also like to thank my institution for providing the necessary resources and learning environment required for completing this project. Special thanks to my friends and family for their encouragement, motivation, and support during the completion of this project. Overall, this project has been a valuable learning experience for me, both academically and personally.

TABLE OF CONTENT

1. Abstract 2
2. Introduction 3
3. Objective of Project 4
4. Methodology / Algorithm 5
5. Code Implementation ..... 6 - 8
6. GUI Implementation ..... 9 - 11
7. Core Java Concepts Used 12
8. Conclusion 13
9. Acknowledgement 14

ABSTRACT

The "Password Strength Checker" is a Java-based mini project designed to evaluate the strength of a user's password and provide suggestions for improvement. In today's digital world, creating secure passwords is essential to protect personal and sensitive information from unauthorized access. This project aims to help users understand how strong their passwords are and how they can enhance security.

The application is developed using Java Swing to provide a simple and user-friendly graphical interface. Users can enter a password, and the system analyzes it based on various parameters such as length, use of uppercase and lowercase letters, numbers, and special characters. Based on these factors, the password is categorized as Weak, Medium, or Strong.

Additionally, the system provides visual feedback using a progress bar and color indicators, along with suggestions to improve the password strength. This project demonstrates the practical implementation of object-oriented programming, GUI development, and logical validation techniques in Java.

Overall, the Password Strength Checker serves as an effective tool for promoting better password practices and enhancing user awareness about cybersecurity.

1. INTRODUCTION

In today's digital era, the use of passwords has become an essential part of securing personal and professional information. From social media accounts to banking systems, passwords act as the first line of defense against unauthorized access. However, many users tend to create weak passwords that are easy to guess, making their data vulnerable to cyber threats.

The "Password Strength Checker" is a Java-based application developed to address this issue by helping users evaluate the strength of their passwords. This project aims to educate users about the importance of creating strong and secure passwords by analyzing different aspects such as password length, use of uppercase and lowercase letters, numbers, and special characters.

The application is designed using Java Swing to provide a simple and interactive graphical user interface. It allows users to input a password and instantly receive feedback on its strength, categorized as Weak, Medium, or Strong. Additionally, the system provides suggestions to improve the password, encouraging users to follow better security practices.

This project not only enhances awareness about cybersecurity but also demonstrates the practical application of programming concepts such as object-oriented programming, event handling, and GUI design in Java. It serves as a useful tool for both learning and real-world application.

2. OBJECTIVE OF THE PROJECT

The main objectives of the "Password Strength Checker" project are as follows:

• To design and develop a user-friendly application that evaluates the strength of a password.

• To help users understand the importance of creating strong and secure passwords for protecting their personal data.

• To analyze passwords based on different parameters such as length, uppercase and lowercase letters, numbers, and special characters.

• To categorize passwords into different levels such as Weak, Medium, and Strong.

• To provide suggestions for improving password strength and enhancing security.

• To implement the concepts of object-oriented programming using Java.

• To develop a graphical user interface (GUI) using Java Swing for better user interaction.

• To gain practical experience in Java programming and real-world problem solving.

• To increase awareness about cybersecurity and safe password practices.

Overall, the objective of this project is to create a simple yet effective tool that promotes better password habits and demonstrates the practical use of programming concepts.

3. METHODOLOGY / ALGORITHM

The "Password Strength Checker" project follows a simple and structured methodology to evaluate the strength of a user's password. The working of the system is based on predefined rules and logical conditions.

Methodology:

The application is developed using Java Swing to create a graphical user interface. The user enters a password in the input field, and upon clicking the "Check" button, the system analyzes the password based on various criteria such as length, characters used, and complexity. The result is then displayed using labels and a progress bar along with color indicators.

Algorithm:

Step 1: Start the program.

Step 2: Create the GUI components such as window, labels, password field, button, and progress bar.

Step 3: Accept the password input from the user.

Step 4: Initialize variables to check different conditions:

- Length of password
- Presence of uppercase letters
- Presence of lowercase letters
- Presence of digits
- Presence of special characters

Step 5: Check the length of the password:

- If length < 6 → Weak
- If length ≥ 6 → proceed to further checks

Step 6: Check character conditions:

- If only letters → Weak
- If letters + numbers → Medium
- If uppercase + lowercase + numbers + special characters and length ≥ 8 → Strong

Step 7: Assign strength level (Weak, Medium, Strong).

Step 8: Update UI:

- Display strength text

- Change color (Red / Orange / Green)
- Update progress bar value

Step 9: Display suggestions:

- Add uppercase letters if missing
- Add numbers if missing
- Add special characters if missing
- Increase length if required

Step 10: End the program.

This methodology ensures that the password is evaluated efficiently and provides instant feedback to the user for improving password security.

4. CODE IMPLEMENTATION

```
import java.util.Scanner;

public class PasswordStrengthChecker {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your password: ");
        String password = sc.nextLine();

        int strength = checkStrength(password);

        if (strength == 1) {
            System.out.println("Weak Password");
        } else if (strength == 2) {
            System.out.println("Medium Password");
        } else {
            System.out.println("Strong Password");
        }

        sc.close();
    }

    public static int checkStrength(String password) {
        int length = password.length();

        boolean hasUpper = password.matches(".*[A-Z].*");
        boolean hasLower = password.matches(".*[a-z].*");
        boolean hasNumber = password.matches(".*[0-9].*");
        boolean hasSpecial = password.matches(".*[@#$%^&+=!].*");

        // Weak
        if (length < 6) {
            return 1;
        }

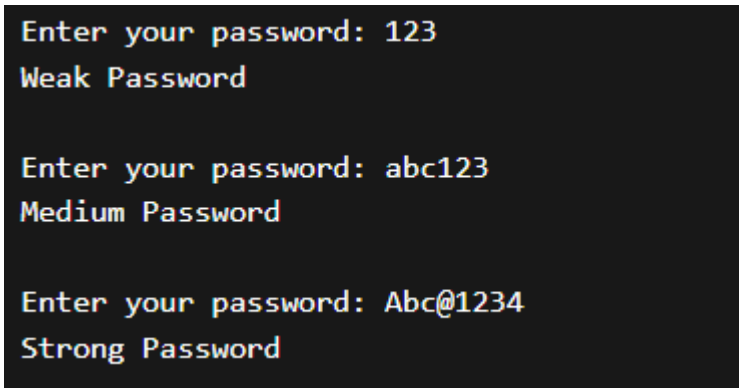
        // Strong
        if (length >= 8 && hasUpper && hasLower && hasNumber && hasSpecial) {
            return 3;
```

```
        }

        // Medium
        if (length >= 6 && (hasLower || hasUpper) && hasNumber) {
            return 2;
        }

        return 1;
    }
}
```

OUTPUT :



5. GUI IMPLEMENTATION

```
import java.awt.*;
import javax.swing.*;

public class PasswordStrengthChecker extends JFrame {
  private JPasswordField passwordField;
  private JTextField textField;
  private JLabel strengthLabel;
  private JProgressBar progressBar;
  private JButton checkButton;
  private JCheckBox showPasswordCheckbox;
  private boolean showPassword = false;
  public PasswordStrengthChecker() {
    // Frame setup
    setTitle("Password Strength Checker");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(500, 380);
    setLocationRelativeTo(null);
    setResizable(false);
    // Main panel with modern gradient background
    JPanel mainPanel = new JPanel() {
      @Override
      protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_RENDERING,
RenderingHints.VALUE_RENDER_QUALITY);
        GradientPaint gradient = new GradientPaint(0, 0, new Color(15, 32, 65),
            0, getHeight(), new Color(32, 58, 110));
        g2d.setPaint(gradient);
        g2d.fillRect(0, 0, getWidth(), getHeight());
      }
    };
    mainPanel.setLayout(new BoxLayout(mainPanel, BoxLayout.Y_AXIS));
    mainPanel.setBorder(BorderFactory.createEmptyBorder(40, 40, 40, 40));
    // Title label
    JLabel titleLabel = new JLabel(" Password Strength Checker");
```

```
        titleLabel.setFont(new Font("Segoe UI", Font.BOLD, 22));
        titleLabel.setForeground(new Color(255, 255, 255));
        titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
        mainPanel.add(titleLabel);
        mainPanel.add(Box.createVerticalStrut(10));
        // Subtitle
        JLabel subtitleLabel = new JLabel("Check your password strength");
        subtitleLabel.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        subtitleLabel.setForeground(new Color(200, 200, 200));
        subtitleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
        mainPanel.add(subtitleLabel);
        mainPanel.add(Box.createVerticalStrut(25));
        // Password field container
        JPanel passwordPanel = new JPanel();
        passwordPanel.setLayout(new BoxLayout(passwordPanel,
    BoxLayout.X_AXIS));
        passwordPanel.setOpaque(false);
        // Password field with border
        passwordField = new JPasswordField();
        passwordField.setFont(new Font("Segoe UI", Font.PLAIN, 14));
        passwordField.setMaximumSize(new Dimension(Integer.MAX_VALUE,
    40));
        passwordField.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(100, 150, 200), 2),
            BorderFactory.createEmptyBorder(8, 12, 8, 12)
        ));
        passwordField.setBackground(new Color(240, 245, 250));
        passwordField.setCaretColor(new Color(30, 100, 200));

        // Hidden text field for showing password
        textField = new JTextField();
        textField.setFont(new Font("Segoe UI", Font.PLAIN, 14));
        textField.setMaximumSize(new Dimension(Integer.MAX_VALUE, 40));
        textField.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(new Color(100, 150, 200), 2),
            BorderFactory.createEmptyBorder(8, 12, 8, 12)
        ));
```

```
        textField.setBackground(new Color(240, 245, 250));
        textField.setCaretColor(new Color(30, 100, 200));
        textField.setVisible(false);
        passwordPanel.add(passwordField);
        passwordPanel.add(Box.createHorizontalStrut(10));
        // Show password checkbox
        showPasswordCheckbox = new JCheckBox("Show");
        showPasswordCheckbox.setFont(new Font("Segoe UI", Font.PLAIN, 12));
        showPasswordCheckbox.setForeground(new Color(200, 200, 255));
        showPasswordCheckbox.setOpaque(false);
        showPasswordCheckbox.setCursor(new Cursor(Cursor.HAND_CURSOR));
        showPasswordCheckbox.addActionListener(e ->
togglePasswordVisibility());
        passwordPanel.add(showPasswordCheckbox);
        mainPanel.add(passwordPanel);
        mainPanel.add(Box.createVerticalStrut(20));
        // Check button with gradient
        checkButton = new JButton("CHECK STRENGTH") {
            @Override
            protected void paintComponent(Graphics g) {
                Graphics2D g2d = (Graphics2D) g;
                g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
                GradientPaint gradient = new GradientPaint(0, 0, new Color(100, 180,
255),
                        0, getHeight(), new Color(60, 140, 220));
                g2d.setPaint(gradient);
                g2d.fillRoundRect(0, 0, getWidth(), getHeight(), 8, 8);
                super.paintComponent(g);
            }
        };
        checkButton.setFont(new Font("Segoe UI", Font.BOLD, 13));
        checkButton.setForeground(Color.WHITE);
        checkButton.setFocusPainted(false);
        checkButton.setBorderPainted(false);
        checkButton.setContentAreaFilled(false);
        checkButton.setPreferredSize(new Dimension(150, 40));
```

```
        checkButton.setMaximumSize(new Dimension(200, 40));
        checkButton.setAlignmentX(Component.CENTER_ALIGNMENT);
        checkButton.setCursor(new Cursor(Cursor.HAND_CURSOR));
        checkButton.addActionListener(e -> checkPasswordStrength());
        mainPanel.add(checkButton);
        mainPanel.add(Box.createVerticalStrut(20));
        // Progress bar
        progressBar = new JProgressBar(0, 100);
        progressBar.setStringPainted(true);
        progressBar.setFont(new Font("Segoe UI", Font.BOLD, 11));
        progressBar.setMaximumSize(new Dimension(Integer.MAX_VALUE, 30));
        progressBar.setBackground(new Color(50, 80, 130));
        progressBar.setForeground(new Color(100, 200, 255));
        progressBar.setBorder(BorderFactory.createLineBorder(new Color(100,
150, 200), 1));
        mainPanel.add(progressBar);
        mainPanel.add(Box.createVerticalStrut(15));
        // Strength label
        strengthLabel = new JLabel("Enter your password and click Check");
        strengthLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));
        strengthLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
        strengthLabel.setForeground(new Color(200, 200, 255));
        mainPanel.add(strengthLabel);
        add(mainPanel);
    }

    private void checkPasswordStrength() {
        String password = showPassword ? textField.getText() : new
String(passwordField.getPassword());

        if (password.isEmpty()) {
            strengthLabel.setText("Please enter a password");
            strengthLabel.setForeground(new Color(200, 200, 255));
            progressBar.setValue(0);
            progressBar.setForeground(new Color(100, 100, 150));
            return;
        }
```

```
    int strength = calculateStrength(password);
    if (strength == 1) {
        strengthLabel.setText("⚠ Weak Password - Too Short");
        strengthLabel.setForeground(new Color(255, 100, 100)); // Bright Red
        progressBar.setValue(33);
        progressBar.setForeground(new Color(255, 80, 80));
    } else if (strength == 2) {
        strengthLabel.setText(" ⚡ Medium Password - Good");
        strengthLabel.setForeground(new Color(255, 200, 80)); // Bright Orange
        progressBar.setValue(66);
        progressBar.setForeground(new Color(255, 200, 80));
    } else {
        strengthLabel.setText("✓ Strong Password - Excellent!");
        strengthLabel.setForeground(new Color(100, 255, 100)); // Bright Green
        progressBar.setValue(100);
        progressBar.setForeground(new Color(100, 255, 100));
    }
}
private void togglePasswordVisibility() {
    showPassword = showPasswordCheckbox.isSelected();

    if (showPassword) {
        // Switch to text field (visible password)
        String password = new String(passwordField.getPassword());
        textField.setText(password);
        passwordField.setVisible(false);
        textField.setVisible(true);
        textField.requestFocus();
        textField.setCaretPosition(textField.getText().length());
    } else {
        // Switch to password field (hidden password)
        String password = textField.getText();
        passwordField.setText(password);
        textField.setVisible(false);
        passwordField.setVisible(true);
        passwordField.requestFocus();
```

```
            passwordField.setCaretPosition(password.length());
        }
    }
    private int calculateStrength(String password) {
        int length = password.length();
    if (length < 6) {
            return 1;
        }
    boolean hasUppercase = password.matches(".*[A-Z].*");
        boolean hasLowercase = password.matches(".*[a-z].*");
        boolean hasNumber = password.matches(".*[0-9].*");
        boolean hasSpecial = password.matches(".*[!@#$%^&*()_+\\-
=\\[\\]{};':\"\\\\|,.<>?].*");

        // Strong: length >= 8 with uppercase, lowercase, number, special character
        if (length >= 8 && hasUppercase && hasLowercase && hasNumber &&
hasSpecial) {
            return 3;
        }

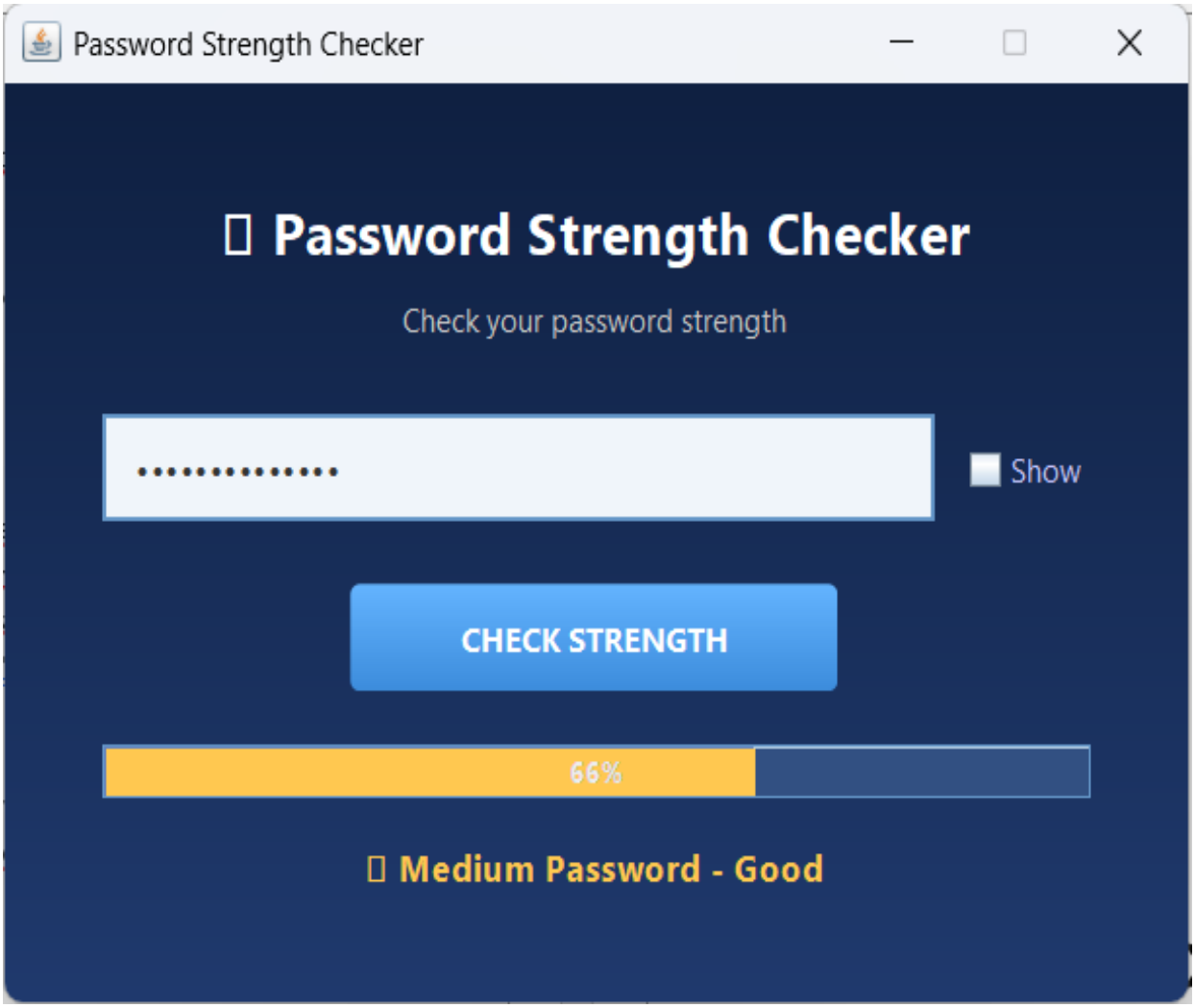
        // Medium: length >= 6 with letters + numbers
        if (hasUppercase || hasLowercase) {
            if (hasNumber) {
                return 2;
            }
        }

        // Default to weak if doesn't meet other criteria
        return 1;
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            PasswordStrengthChecker frame = new PasswordStrengthChecker();
            frame.setVisible(true);
        });
    }
```

}

OUTPUT :


Password Strength Checker
Password Strength Checker
Check your password strength
Show
CHECK STRENGTH
66%
Medium Password - Good

CORE JAVA CONCEPTS USED

The "Password Strength Checker" project is developed using fundamental Core Java concepts. The following concepts are used in the implementation of this project:

• Classes and Objects:
The entire application is built using a class (PasswordStrengthChecker), which represents the structure of the program. Objects are used to access methods and execute functionality.

• Methods (Functions):
Methods such as checkPasswordStrength() and calculateStrength() are used to divide the program into smaller, manageable parts and perform specific tasks.

• Conditional Statements (if-else):
Decision-making is implemented using if-else statements to classify passwords as Weak, Medium, or Strong based on different conditions.

• Loops (if applicable):
Logical checks are performed repeatedly when needed, especially in validation (though explicit loops may not be heavily used in GUI version).

• String Handling:
String methods and pattern matching (using matches()) are used to analyze password characters such as uppercase, lowercase, digits, and special symbols.

• Regular Expressions (Regex):
Regex is used to check the presence of specific character types in the password (e.g., numbers, special characters).

• Java Swing (GUI Programming):
Swing components such as JFrame, JLabel, JButton, JPasswordField, JProgressBar, and JCheckBox are used to create an interactive graphical user interface.

• Event Handling:
ActionListener is used to handle button clicks and user interactions.

• Encapsulation:
Variables and methods are organized within a class to maintain proper structure and data security.

• Exception Handling (basic awareness):
Although not explicitly used, the program can be extended to handle invalid inputs using try-catch blocks.

Overall, this project demonstrates the practical use of Core Java concepts to build a real-world application with both logic and user interface.

8. Conclusion

The "Password Strength Checker" project has been successfully developed using Core Java concepts and Java Swing for graphical user interface design. This project provides an effective way to evaluate the strength of a password based on important factors such as length, use of uppercase and lowercase letters, numbers, and special characters.

Through this project, we were able to understand how programming concepts like object-oriented programming, event handling, and string processing can be applied to solve real-world problems. The application not only checks password strength but also helps users become aware of the importance of creating secure passwords.

The graphical interface makes the application user-friendly and easy to use, while the logic ensures accurate and quick evaluation of passwords. Overall, this project has enhanced our practical knowledge of Java programming and improved our problem-solving skills.

In conclusion, the Password Strength Checker is a simple yet useful tool that promotes better cybersecurity practices and demonstrates the effective implementation of Java in real-life applications.